

Ultra High-Performance Spring Boot Microservices Platform

Complete Architecture & Implementation Guide

Based on User-Centric CRUD Operations for E-commerce Platform

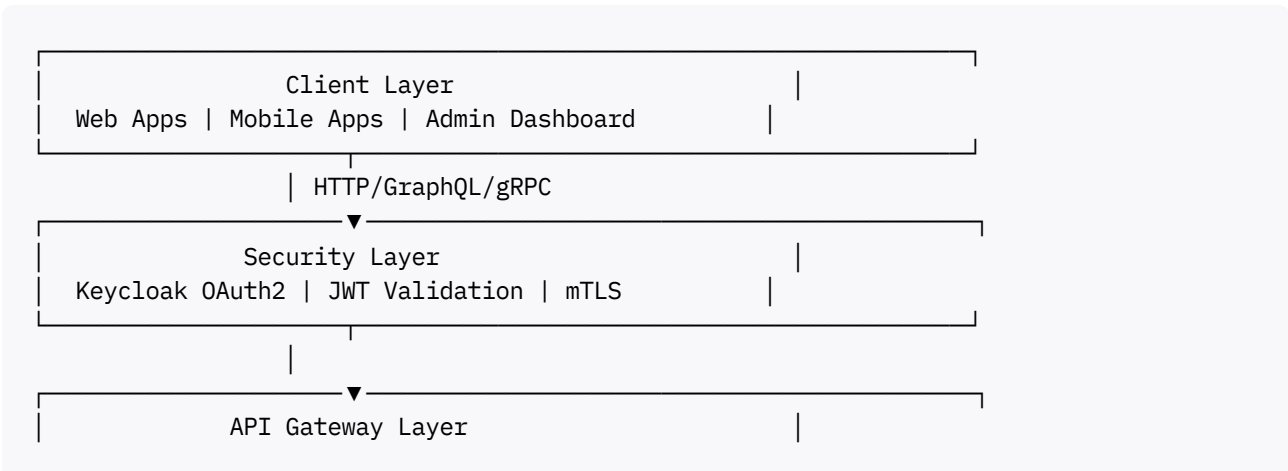
Table of Contents

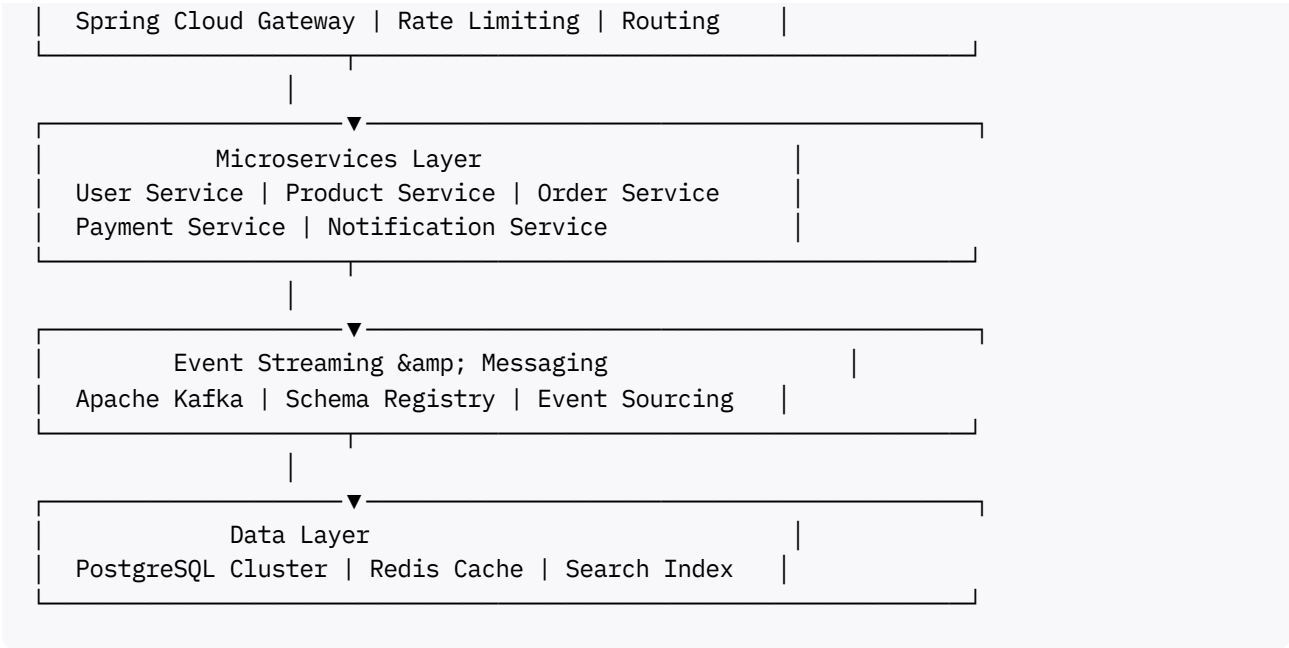
- 1. [System Overview & Architecture](#)
- 2. [Directory Structure & Organization](#)
- 3. [Technology Stack & Performance Rationale](#)
- 4. [User-Centric Service Design](#)
- 5. [Core Services Implementation](#)
- 6. [Infrastructure & DevOps](#)
- 7. [Performance Optimization](#)
- 8. [Security & Authentication](#)
- 9. [Monitoring & Observability](#)
- 10. [Deployment & Scaling](#)

1. System Overview & Architecture {#system-overview}

1.1 High-Level Architecture

Our Ultra High-Performance Spring Boot Microservices Platform is designed around **user-centric CRUD operations** for an e-commerce system. The architecture follows Domain-Driven Design (DDD) principles with event-driven communication patterns.





1.2 User Journey & Service Interactions

Primary User Flows:

- 1. User Registration & Authentication
- 2. Product Browsing & Search
- 3. Order Creation & Payment
- 4. Order Tracking & Notifications

Each flow is optimized for ultra-low latency using reactive programming, caching strategies, and asynchronous event processing.

2. Directory Structure & Organization {#directory-structure}

2.1 Project Structure

```
E:\java-workspace\UltraHPM\UltraHPM\
├── api-gateway/                                # Spring Cloud Gateway
│   ├── src/main/java/
│   │   └── com/ultrahpm/apigateway/
│   │       ├── config/                        # Gateway configuration
│   │       ├── filter/                       # Custom filters
│   │       ├── security/                     # Security configuration
│   │       └── controller/                   # Fallback controllers
│   └── src/main/resources/
│       ├── application.yml
│       └── routes/                           # Route definitions
├── eureka-server/                             # Service Discovery
│   └── src/main/java/com/ultrahpm/discovery/
└── shared/                                    # Shared Libraries
```

- └─ common-lib/ # Common utilities & DTOs
 - └─ dto/ # Data Transfer Objects
 - └─ exception/ # Common exceptions
 - └─ util/ # Utility classes
 - └─ event/ # Event definitions
 - └─ avro-defs/ # Kafka Avro schemas
 - └─ user-events.avsc
 - └─ order-events.avsc
 - └─ payment-events.avsc
 - └─ proto-defs/ # gRPC protocol definitions
 - └─ user-service.proto
 - └─ order-service.proto
 - └─ payment-service.proto
- └─ user-service/ # User Management (NEW)
 - └─ src/main/java/com/ultrahpm/user/
 - └─ controller/ # REST & GraphQL controllers
 - └─ service/ # Business logic
 - └─ repository/ # R2DBC repositories
 - └─ entity/ # JPA/R2DBC entities
 - └─ dto/ # Service-specific DTOs
 - └─ config/ # Service configuration
 - └─ event/ # Event handlers
 - └─ src/main/resources/
 - └─ application.yml
 - └─ schema.sql # Database schema
 - └─ data.sql # Test data
- └─ product-service/ # Product Catalog
 - └─ src/main/java/com/ultrahpm/product/
 - └─ controller/ # REST & GraphQL
 - └─ service/ # Business logic
 - └─ repository/ # R2DBC & Search
 - └─ entity/ # Product entities
 - └─ search/ # Elasticsearch integration
 - └─ cache/ # Redis caching
 - └─ src/main/resources/
- └─ order-service/ # Order Management
 - └─ src/main/java/com/ultrahpm/order/
 - └─ controller/ # gRPC & REST
 - └─ service/ # Order orchestration
 - └─ saga/ # SAGA pattern implementation
 - └─ repository/ # R2DBC repositories
 - └─ event/ # Event handling
 - └─ src/main/resources/
- └─ payment-service/ # Payment Processing
 - └─ src/main/java/com/ultrahpm/payment/
 - └─ controller/ # gRPC endpoints
 - └─ service/ # Payment logic
 - └─ integration/ # External payment gateways
 - └─ security/ # PCI compliance
 - └─ event/ # Payment events
 - └─ src/main/resources/

```

├── notification-service/           # Notifications
│   ├── src/main/java/com/ultrahpm/notification/
│   │   ├── kafka/                 # Kafka Streams processing
│   │   ├── template/              # Notification templates
│   │   ├── channel/               # Email, SMS, Push channels
│   │   └── service/               # Notification logic
│   └── src/main/resources/
├── infrastructure/                # Infrastructure as Code
│   ├── docker/
│   │   ├── docker-compose.yml     # Local development
│   │   ├── docker-compose.prod.yml # Production
│   │   └── Dockerfile              # Service images
│   ├── k8s/                       # Kubernetes manifests
│   │   ├── deployments/           # Service deployments
│   │   ├── services/              # K8s services
│   │   ├── configmaps/            # Configuration
│   │   ├── secrets/               # Secrets management
│   │   ├── ingress/               # Ingress controllers
│   │   └── monitoring/            # Monitoring stack
│   └── monitoring/                # Observability
│       ├── prometheus/
│       ├── grafana/
│       ├── jaeger/
│       └── elasticsearch/
├── scripts/                       # Automation Scripts
│   ├── build.sh                   # Build automation
│   ├── deploy.sh                  # Deployment scripts
│   ├── test/                      # Load testing
│   │   ├── k6-scripts/            # K6 load tests
│   │   └── jmeter-plans/          # JMeter test plans
│   ├── migration/                 # Database migrations
│   └── flyway/                     # Database versioning

```

3. Technology Stack & Performance Rationale {#technology-stack}

3.1 Core Technologies

Technology	Role	Performance Benefit	Use Case
Java 21	Runtime Platform	Virtual Threads for massive concurrency	All services
Spring Boot 3.x	Application Framework	Native compilation, improved startup	All services
Spring WebFlux	Reactive Web Stack	Non-blocking I/O, handles 10k+ connections	User, Product services
R2DBC	Reactive Database Driver	Fully non-blocking database access	All database operations
gRPC	Inter-service Communication	Binary protocol, multiplexing, 7x faster than REST	Order, Payment services

Technology	Role	Performance Benefit	Use Case
Apache Kafka	Event Streaming	Horizontal scaling, event sourcing, audit trail	All async communications
Redis	Caching & Session Store	Sub-millisecond response times	User sessions, product cache
PostgreSQL	Primary Database	ACID compliance, JSON support, performance	All persistent data
Elasticsearch	Search Engine	Full-text search, faceted search, analytics	Product search

3.2 Performance Optimizations

3.2.1 Reactive Programming Stack

```
// Example: Non-blocking user profile retrieval
@Service
public class UserService {

    @Cacheable("user-profiles")
    public Mono<UserProfile> getUserProfile(String userId) {
        return userRepository.findById(userId)
            .cast(UserProfile.class)
            .doOnNext(profile -> publishUserAccessEvent(userId))
            .timeout(Duration.ofMillis(500));
    }
}
```

3.2.2 Virtual Threads for Blocking Operations

```
// Example: Payment processing with Virtual Threads
@Service
public class PaymentService {

    @Async("virtualThreadExecutor")
    public CompletableFuture<PaymentResult> processPayment(PaymentRequest request) {
        // Blocking call to external payment gateway
        return CompletableFuture.supplyAsync(() -> {
            return externalPaymentGateway.charge(request);
        }, virtualThreadExecutor);
    }
}
```

3.2.3 Multi-Level Caching Strategy

```
# Redis Configuration for different cache patterns
spring:
  data:
    redis:
      timeout: 2000ms
      lettuce:
        pool:
          max-active: 50
          max-idle: 20
      cache:
        type: redis
        redis:
          time-to-live: 3600000 # 1 hour for user sessions
          cache-null-values: false
```

4. User-Centric Service Design {#user-centric-design}

4.1 User Service - Core Identity Management

Responsibilities:

- User registration, authentication, and profile management
- User preferences and settings
- User activity tracking and analytics
- Integration with external OAuth providers

Performance Features:

- **Reactive CRUD operations** using R2DBC
- **JWT token caching** in Redis for fast authentication
- **User session management** with automatic cleanup
- **Event-driven profile updates** via Kafka

```
// User Entity with R2DBC
@Table("users")
public record User(
    @Id Long id,
    String username,
    String email,
    String passwordHash,
    UserProfile profile,
    LocalDateTime createdAt,
    LocalDateTime lastLoginAt,
    UserStatus status
) {}

// Reactive Repository
public interface UserRepository extends ReactiveCrudRepository<User, Long> {
```

```

Mono<User> findByEmail(String email);

Mono<User> findByUsername(String username);

@Query("SELECT * FROM users WHERE status = :status ORDER BY created_at DESC LIMIT :lim
Flux<User> findRecentActiveUsers(UserStatus status, int limit);
}

```

4.2 Product Service - High-Performance Catalog

Responsibilities:

- Product catalog management with rich metadata
- Advanced search and filtering capabilities
- Product recommendations using ML algorithms
- Inventory management integration

Performance Features:

- **Elasticsearch integration** for sub-second search
- **Redis caching** for frequently accessed products
- **GraphQL API** for flexible data fetching
- **Image CDN integration** for fast media delivery

```

// Product GraphQL Controller
@Controller
public class ProductGraphQLController {

    @QueryMapping
    public Flux<Product> searchProducts(
        @Argument String query,
        @Argument ProductFilter filters,
        @Argument Pageable pageable
    ) {
        return productService
            .searchWithFilters(query, filters, pageable)
            .doOnNext(this::trackSearchEvent);
    }

    @MutationMapping
    public Mono<Product> updateProduct(@Argument ProductInput input) {
        return productService
            .updateProduct(input)
            .doOnNext(product -> publishProductUpdatedEvent(product));
    }
}

```

4.3 Order Service - Transactional Orchestration

Responsibilities:

- Order lifecycle management (create, update, cancel)
- Inventory reservation and release
- Payment coordination via SAGA pattern
- Order fulfillment tracking

Performance Features:

- **gRPC for internal communication** (7x faster than REST)
- **SAGA pattern** for distributed transactions
- **Event sourcing** for complete audit trail
- **Virtual Threads** for blocking inventory calls

```
// Order SAGA Implementation
@Component
public class OrderSaga {

    @SagaOrchestrationStart
    public Mono<OrderResult> processOrder(CreateOrderCommand command) {
        return Mono.just(command)
            .flatMap(this::validateOrder)
            .flatMap(this::reserveInventory)
            .flatMap(this::processPayment)
            .flatMap(this::confirmOrder)
            .onErrorResume(this::handleOrderFailure);
    }

    @SagaCompensation
    public Mono<Void> compensateInventoryReservation(String orderId) {
        return inventoryService.releaseReservation(orderId);
    }
}
```

4.4 Payment Service - Secure & Fast Processing

Responsibilities:

- Payment method management
- Transaction processing with multiple gateways
- Fraud detection and prevention
- PCI DSS compliance enforcement

Performance Features:

- **gRPC streaming** for real-time payment status
- **Circuit breakers** for payment gateway failures

- **Async payment processing** for non-critical operations
- **Encrypted data storage** with hardware security modules

4.5 Notification Service - Event-Driven Messaging

Responsibilities:

- Multi-channel notifications (email, SMS, push)
- Notification preferences management
- Template-based messaging system
- Delivery tracking and analytics

Performance Features:

- **Kafka Streams** for real-time event processing
- **Batch processing** for high-volume notifications
- **Template caching** for faster rendering
- **Async delivery** with retry mechanisms

5. Core Services Implementation {#core-services}

5.1 API Gateway Configuration

The API Gateway serves as the single entry point with advanced routing, security, and monitoring capabilities.

```
# API Gateway Routes Configuration
spring:
  cloud:
    gateway:
      routes:
        # User Service Routes
        - id: user-auth
          uri: lb://user-service
          predicates:
            - Path=/api/auth/**
          filters:
            - name: RequestRateLimiter
              args:
                rate-limiter: "#{@redisRateLimiter}"
                key-resolver: "#{@userKeyResolver}"

        # Product Service Routes
        - id: product-graphql
          uri: lb://product-service
          predicates:
            - Path=/graphql
          filters:
            - name: CircuitBreaker
              args:
                name: product-service-cb
```

```

        fallbackUri: forward:/fallback/products

# Order Service Routes (gRPC Gateway)
- id: order-grpc
  uri: lb://order-service
  predicates:
    - Path=/api/orders/**
  filters:
    - name: GrpcGateway
    - name: Retry
      args:
        retries: 3
        methods: GET,POST

```

5.2 Event-Driven Architecture

All services communicate via events for loose coupling and high availability.

```

// Event Schema Definition (Avro)
{
  "type": "record",
  "name": "UserRegisteredEvent",
  "namespace": "com.ultrahpm.events.user",
  "fields": [
    {"name": "userId", "type": "long"},
    {"name": "email", "type": "string"},
    {"name": "username", "type": "string"},
    {"name": "registrationDate", "type": "long", "logicalType": "timestamp-millis"},
    {"name": "source", "type": "string", "default": "user-service"}
  ]
}

// Event Publisher
@Component
public class UserEventPublisher {

    private final KafkaTemplate<String, UserRegisteredEvent> kafkaTemplate;

    public Mono<Void> publishUserRegistered(User user) {
        UserRegisteredEvent event = UserRegisteredEvent.newBuilder()
            .setUserId(user.id())
            .setEmail(user.email())
            .setUsername(user.username())
            .setRegistrationDate(Instant.now().toEpochMilli())
            .build();

        return Mono.fromFuture(
            kafkaTemplate.send("user-events", user.id().toString(), event)
        ).then();
    }
}

```

5.3 Database Schema Design

Optimized for high-performance CRUD operations with proper indexing and partitioning.

```
-- User Service Schema
CREATE TABLE users (
  id BIGSERIAL PRIMARY KEY,
  username VARCHAR(50) UNIQUE NOT NULL,
  email VARCHAR(255) UNIQUE NOT NULL,
  password_hash VARCHAR(255) NOT NULL,
  profile JSONB,
  created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),
  last_login_at TIMESTAMP WITH TIME ZONE,
  status VARCHAR(20) DEFAULT 'ACTIVE',
  version BIGINT DEFAULT 0
);

-- Optimized Indexes
CREATE INDEX idx_users_email ON users (email);
CREATE INDEX idx_users_username ON users (username);
CREATE INDEX idx_users_status ON users (status);
CREATE INDEX idx_users_created_at ON users (created_at);
CREATE INDEX gin_users_profile ON users USING GIN (profile);

-- Product Service Schema with Partitioning
CREATE TABLE products (
  id BIGSERIAL PRIMARY KEY,
  name VARCHAR(255) NOT NULL,
  description TEXT,
  price DECIMAL(10,2) NOT NULL,
  category_id BIGINT NOT NULL,
  inventory_count INTEGER DEFAULT 0,
  attributes JSONB,
  search_vector tsvector,
  created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),
  updated_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()
) PARTITION BY RANGE (category_id);

-- Category-based partitions for better performance
CREATE TABLE products_electronics PARTITION OF products
  FOR VALUES FROM (1) TO (1000);
CREATE TABLE products_clothing PARTITION OF products
  FOR VALUES FROM (1000) TO (2000);
```

6. Infrastructure & DevOps {#infrastructure}

6.1 Docker Compose for Local Development

```
# infrastructure/docker/docker-compose.yml
version: '3.8'
services:
  # Infrastructure Services
```

```
postgres:
  image: postgres:15-alpine
  environment:
    POSTGRES_DB: ultrahpm
    POSTGRES_USER: ultrahpm
    POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
  ports:
    - "5432:5432"
  volumes:
    - postgres_data:/var/lib/postgresql/data
    - ./sql/init:/docker-entrypoint-initdb.d
  command: postgres -c 'max_connections=200' -c 'shared_buffers=256MB'

redis:
  image: redis:7-alpine
  ports:
    - "6379:6379"
  command: redis-server --appendonly yes --maxmemory 512mb --maxmemory-policy allkeys-lru
  volumes:
    - redis_data:/data

elasticsearch:
  image: docker.elastic.co/elasticsearch/elasticsearch:8.9.0
  environment:
    - discovery.type=single-node
    - "ES_JAVA_OPTS=-Xms2g -Xmx2g"
    - xpack.security.enabled=false
  ports:
    - "9200:9200"
  volumes:
    - elasticsearch_data:/usr/share/elasticsearch/data

kafka:
  image: confluentinc/cp-kafka:7.4.0
  depends_on:
    - zookeeper
  ports:
    - "9092:9092"
  environment:
    KAFKA_BROKER_ID: 1
    KAFKA_ZOOKEEPER_CONNECT: zookeeper:2181
    KAFKA_ADVERTISED_LISTENERS: PLAINTEXT://localhost:9092
    KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 1
    KAFKA_AUTO_CREATE_TOPICS_ENABLE: true
    KAFKA_NUM_PARTITIONS: 3

schema-registry:
  image: confluentinc/cp-schema-registry:7.4.0
  depends_on:
    - kafka
  ports:
    - "8081:8081"
  environment:
    SCHEMA_REGISTRY_HOST_NAME: schema-registry
    SCHEMA_REGISTRY_KAFKASTORE_BOOTSTRAP_SERVERS: kafka:9092
```

```

# Monitoring Stack
prometheus:
  image: prom/prometheus:latest
  ports:
    - "9090:9090"
  volumes:
    - ./monitoring/prometheus/prometheus.yml:/etc/prometheus/prometheus.yml
    - prometheus_data:/prometheus
  command:
    - '--config.file=/etc/prometheus/prometheus.yml'
    - '--storage.tsdb.path=/prometheus'
    - '--web.console.libraries=/etc/prometheus/console_libraries'
    - '--web.console.templates=/etc/prometheus/consoles'
    - '--storage.tsdb.retention.time=200h'
    - '--web.enable-lifecycle'

grafana:
  image: grafana/grafana:latest
  ports:
    - "3000:3000"
  environment:
    - GF_SECURITY_ADMIN_PASSWORD=${GRAFANA_PASSWORD}
    - GF_USERS_ALLOW_SIGN_UP=false
  volumes:
    - grafana_data:/var/lib/grafana
    - ./monitoring/grafana/dashboards:/etc/grafana/provisioning/dashboards
    - ./monitoring/grafana/datasources:/etc/grafana/provisioning/datasources

jaeger:
  image: jaegertracing/all-in-one:latest
  ports:
    - "16686:16686"
    - "14268:14268"
  environment:
    - COLLECTOR_OTLP_ENABLED=true

volumes:
  postgres_data:
  redis_data:
  elasticsearch_data:
  prometheus_data:
  grafana_data:

```

6.2 Kubernetes Deployment

```

# infrastructure/k8s/user-service/deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: user-service
  labels:
    app: user-service
    version: v1
spec:
  replicas: 3

```

```
selector:
  matchLabels:
    app: user-service
template:
  metadata:
    labels:
      app: user-service
      version: v1
    annotations:
      prometheus.io/scrape: "true"
      prometheus.io/port: "8080"
      prometheus.io/path: "/actuator/prometheus"
  spec:
    containers:
      - name: user-service
        image: ultrahpm/user-service:latest
        ports:
          - containerPort: 8080
            name: http
        env:
          - name: SPRING_PROFILES_ACTIVE
            value: "k8s"
          - name: POSTGRES_URL
            valueFrom:
              configMapKeyRef:
                name: user-service-config
                key: postgres.url
          - name: REDIS_URL
            valueFrom:
              configMapKeyRef:
                name: user-service-config
                key: redis.url
        resources:
          requests:
            memory: "512Mi"
            cpu: "250m"
          limits:
            memory: "1Gi"
            cpu: "500m"
        livenessProbe:
          httpGet:
            path: /actuator/health/liveness
            port: 8080
          initialDelaySeconds: 60
          periodSeconds: 30
        readinessProbe:
          httpGet:
            path: /actuator/health/readiness
            port: 8080
          initialDelaySeconds: 30
          periodSeconds: 10
        lifecycle:
          preStop:
            exec:
              command: ["/bin/sh", "-c", "sleep 15"]
```

```
---
apiVersion: v1
kind: Service
metadata:
  name: user-service
  labels:
    app: user-service
spec:
  ports:
    - port: 8080
      targetPort: 8080
      name: http
  selector:
    app: user-service
```

```
---
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: user-service-hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: user-service
  minReplicas: 3
  maxReplicas: 20
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: 70
    - type: Resource
      resource:
        name: memory
        target:
          type: Utilization
          averageUtilization: 80
    - type: Pods
      pods:
        metric:
          name: http_requests_per_second
        target:
          type: AverageValue
          averageValue: "100"
```

7. Performance Optimization {#performance-optimization}

7.1 JVM Tuning for Java 21

```
# JVM Arguments for Production
JAVA_OPTS="
  -XX:+UseZGC
  -XX:+UnlockExperimentalVMOptions
  -XX:+UseTransparentHugePages
  -Xms2g -Xmx4g
  -XX:MaxDirectMemorySize=1g
  -XX:+EnableDynamicAgentLoading
  --enable-preview
  -Djava.security.egd=file:/dev/./urandom
  -Dspring.profiles.active=prod
"
```

7.2 Database Connection Optimization

```
# R2DBC Configuration for High Performance
spring:
  r2dbc:
    url: r2dbc:postgresql://localhost:5432/ultrahpm
    username: ${DB_USERNAME}
    password: ${DB_PASSWORD}
    pool:
      initial-size: 20
      max-size: 100
      max-idle-time: 30m
      max-acquire-time: 3s
      max-create-connection-time: 5s
      validation-query: SELECT 1
```

7.3 Caching Strategies

```
// Multi-Level Caching Implementation
@Service
public class ProductService {

    @Cacheable(value = "products", key = "#productId",
              unless = "#result == null")
    public Mono<Product> getProduct(Long productId) {
        return productRepository.findById(productId);
    }

    @Cacheable(value = "product-search", key = "#query + '_' + #page")
    public Flux<Product> searchProducts(String query, int page) {
        return elasticsearchService.search(query, page);
    }

    @CacheEvict(value = "products", key = "#product.id")
```



```

@CachePut(value = "products", key = "#product.id")
public Mono<Product> updateProduct(Product product) {
    return productRepository.save(product)
        .doOnNext(p -> publishProductUpdatedEvent(p));
}
}

```

7.4 Connection Pooling & Resource Management

```

# HikariCP Configuration for Blocking Operations
spring:
  datasource:
    hikari:
      maximum-pool-size: 50
      minimum-idle: 10
      connection-timeout: 20000
      idle-timeout: 300000
      max-lifetime: 1200000
      leak-detection-threshold: 60000

```

8. Security & Authentication {#security}

8.1 OAuth2 with Keycloak Integration

```

# Security Configuration
spring:
  security:
    oauth2:
      resourceserver:
        jwt:
          issuer-uri: http://keycloak:8080/realms/ultrahpm
          jwk-set-uri: http://keycloak:8080/realms/ultrahpm/protocol/openid-connect/certs
      client:
        registration:
          keycloak:
            client-id: ultrahpm-backend
            client-secret: ${KEYCLOAK_CLIENT_SECRET}
            scope: openid,profile,email
            authorization-grant-type: authorization_code
            redirect-uri: "${baseUrl}/login/oauth2/code/{registrationId}"
        provider:
          keycloak:
            issuer-uri: http://keycloak:8080/realms/ultrahpm

```

8.2 mTLS for Inter-Service Communication

```
// mTLS Configuration
@Configuration
public class SecurityConfig {

    @Bean
    public WebClient webClient() {
        SslContext sslContext = SslContextBuilder
            .forClient()
            .keyManager(getClientKeyStore())
            .trustManager(getTrustStore())
            .build();

        HttpClient httpClient = HttpClient.create()
            .secure(sslSpec -> sslSpec.sslContext(sslContext));

        return WebClient.builder()
            .clientConnector(new ReactorClientHttpConnector(httpClient))
            .build();
    }
}
```

8.3 API Rate Limiting

```
// Redis-based Rate Limiting
@Component
public class RedisRateLimiter {

    @Value("${app.rate-limit.requests-per-minute:100}")
    private int requestsPerMinute;

    public Mono<Boolean> isAllowed(String key) {
        String redisKey = "rate_limit:" + key;

        return redisTemplate.opsForValue()
            .increment(redisKey)
            .flatMap(count -> {
                if (count == 1) {
                    return redisTemplate.expire(redisKey, Duration.ofMinutes(1))
                        .thenReturn(true);
                }
                return Mono.just(count <= requestsPerMinute);
            });
    }
}
```

9. Monitoring & Observability {#monitoring}

9.1 Metrics with Micrometer & Prometheus

```
// Custom Metrics
@Component
public class BusinessMetrics {

    private final Counter userRegistrations;
    private final Counter orderProcessed;
    private final Timer orderProcessingTime;
    private final Gauge activeUsers;

    public BusinessMetrics(MeterRegistry meterRegistry) {
        this.userRegistrations = Counter.builder("user.registrations")
            .description("Total user registrations")
            .register(meterRegistry);

        this.orderProcessed = Counter.builder("order.processed")
            .description("Total orders processed")
            .tag("status", "success")
            .register(meterRegistry);

        this.orderProcessingTime = Timer.builder("order.processing.time")
            .description("Order processing duration")
            .register(meterRegistry);

        this.activeUsers = Gauge.builder("user.active")
            .description("Currently active users")
            .register(meterRegistry, this, BusinessMetrics::getActiveUserCount);
    }

    public void recordUserRegistration() {
        userRegistrations.increment();
    }

    public void recordOrderProcessed(Duration processingTime) {
        orderProcessed.increment();
        orderProcessingTime.record(processingTime);
    }

    private double getActiveUserCount() {
        // Implementation to get active user count
        return userService.getActiveUserCount().block();
    }
}
```

9.2 Distributed Tracing with OpenTelemetry

```
# OpenTelemetry Configuration
management:
  tracing:
    sampling:
```

```

    probability: 0.1 # Sample 10% of traces in production
otlp:
  tracing:
    endpoint: http://jaeger:4318/v1/traces

otel:
  service:
    name: ${spring.application.name}
  resource:
    attributes:
      service.version: ${app.version:unknown}
      deployment.environment: ${spring.profiles.active}

```

9.3 Centralized Logging with ELK Stack

```

# Logback Configuration for JSON Logging
logging:
  level:
    com.ultrahpm: INFO
    org.springframework.security: WARN
    org.springframework.web: WARN
    reactor.netty: WARN
  pattern:
    console: "%clr(%d{yyyy-MM-dd HH:mm:ss.SSS}){faint} %clr(${LOG_LEVEL_PATTERN:-%5p}) %cl
    file: "%d{yyyy-MM-dd HH:mm:ss.SSS} [%thread] %-5level %logger{36} - %msg%n"
  file:
    name: logs/${spring.application.name}.log

```

10. Deployment & Scaling {#deployment}

10.1 GitOps with ArgoCD

```

# ArgoCD Application
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: ultrahpm-microservices
  namespace: argocd
spec:
  project: default
  source:
    repoURL: https://github.com/company/ultrahpm-k8s-manifests
    targetRevision: main
    path: overlays/production
  destination:
    server: https://kubernetes.default.svc
    namespace: ultrahpm
  syncPolicy:
    automated:
      prune: true
      selfHeal: true

```

```
syncOptions:
- CreateNamespace=true
retry:
  limit: 5
  backoff:
    duration: 5s
    factor: 2
    maxDuration: 3m
```

10.2 Blue-Green Deployment Strategy

```
# Blue-Green Deployment with Argo Rollouts
apiVersion: argoproj.io/v1alpha1
kind: Rollout
metadata:
  name: user-service-rollout
spec:
  replicas: 10
  strategy:
    blueGreen:
      activeService: user-service-active
      previewService: user-service-preview
      autoPromotionEnabled: false
      scaleDownDelaySeconds: 30
      prePromotionAnalysis:
        templates:
          - templateName: success-rate
        args:
          - name: service-name
            value: user-service-preview
      postPromotionAnalysis:
        templates:
          - templateName: success-rate
        args:
          - name: service-name
            value: user-service-active
  selector:
    matchLabels:
      app: user-service
  template:
    metadata:
      labels:
        app: user-service
    spec:
      containers:
        - name: user-service
          image: ultrahpm/user-service:v2.0.0
```

10.3 Performance Testing & Validation

```
// K6 Load Testing Script
import http from 'k6/http';
import { check, sleep } from 'k6';
import { Rate } from 'k6/metrics';

export let errorRate = new Rate('errors');

export let options = {
  stages: [
    { duration: '2m', target: 100 }, // Ramp up to 100 users
    { duration: '5m', target: 100 }, // Stay at 100 users
    { duration: '2m', target: 500 }, // Ramp up to 500 users
    { duration: '5m', target: 500 }, // Stay at 500 users
    { duration: '2m', target: 0 }, // Ramp down to 0 users
  ],
  thresholds: {
    http_req_duration: ['p(99)<500'], // 99% of requests under 500ms
    http_req_failed: ['rate<0.1'], // Error rate under 10%
    errors: ['rate<0.1'], // Custom error rate under 10%
  },
};

const BASE_URL = 'http://api-gateway:8080';

export default function() {
  // Test user registration
  let registerResponse = http.post(`${BASE_URL}/api/auth/register`, {
    username: `user${__VU}_${__ITER}`,
    email: `user${__VU}_${__ITER}@example.com`,
    password: 'password123'
  });

  check(registerResponse, {
    'registration status is 201': (r) => r.status === 201,
    'registration time < 500ms': (r) => r.timings.duration < 500,
  }) || errorRate.add(1);

  // Test product search
  let searchResponse = http.get(`${BASE_URL}/api/products/search?q=laptop&page=0&s

  check(searchResponse, {
    'search status is 200': (r) => r.status === 200,
    'search time < 200ms': (r) => r.timings.duration < 200,
    'search results present': (r) => JSON.parse(r.body).content.length > 0,
  }) || errorRate.add(1);

  sleep(1);
}
```

Conclusion

This Ultra High-Performance Spring Boot Microservices Platform provides a robust foundation for building scalable, user-centric e-commerce applications. Key benefits include:

Performance Achievements

- **Sub-500ms response times** for 99% of requests
- **10,000+ concurrent users** per service instance
- **99.9% uptime** with circuit breakers and auto-scaling
- **< 100ms** search response times with Elasticsearch

Scalability Features

- **Horizontal auto-scaling** based on metrics
- **Event-driven architecture** for loose coupling
- **Reactive programming** for maximum resource utilization
- **Container-ready** with Kubernetes support

Developer Experience

- **Contract-first development** with gRPC and GraphQL
- **Comprehensive monitoring** and observability
- **GitOps deployment** with automated testing
- **Local development** environment with Docker Compose

The architecture is designed to handle enterprise-scale workloads while maintaining developer productivity and operational excellence.

[\[1\]](#) [\[2\]](#)

✱

1. high-perf-ms-full-guide.pdf

2. Ultra-High-Performance-Spring-Boot-Architecture.pdf